19 費氏堆積 解答

19-E01 extract-min 根串列

推導

移除根2後，7與5升為root，parent=NIL、mark=false。兩根degree都0，碰撞時令較大鍵7接到5下，5的degree變1。

結論與檢核

所有舊其他 roots 與 children 7、5 形成待整理根列。consolidate 合同度根並在掃描保留根時重建 min；不需要聲稱一定在合併之前完成新的 min 掃描。最後 min 必須是保留根的最小鍵。

19-E02 cascade cut

推導

先令 x 的父 p 不再把 x 指為 child：cut x，x.parent=NIL，x.mark=false；p 原已 marked，因再失一個 child，cut p，p.parent=NIL，p.mark=false。此時分支取決於 g：若 g 非根且未 marked，令 g.mark=true 並停止；若 g 是根，停止且保持 g.mark=false。兩種分支都不得把 g 的 mark 合併成同一結論。

結論與檢核

非根分支：root list 新增 x、p；g 仍是非根父，mark(g)=true。根分支：root list 新增 x、p；g 仍是 root，mark(g)=false。共通部分是 x、p 的 parent 已改為 NIL、mark 清除，後續從 g 以上的父子與標記關係未再改動。

19-E03 degree 最小子樹

推導

degree 5 時 size>=F_7。F_3=2、F_4=3、F_5=5、F_6=8、F_7=13，所以至少 13 個節點。

結論與檢核

這個遞迴下界來自失子歷史允許的 degree 最小規模，呈 Fibonacci 成長。由指數成長反解可限制最大 degree 為 O(lg n)。

19 費氏堆積 解答 （續）

19-E04 級聯切割的總帳

推導

第一個x因key小於父而cut，不必已標記；但後續c−1個級聯cut節點都已標記，各清除一個mark。最後最多新標記一個非根，所以Delta m<=−(c−1)+1=2−c，Delta t=c，故Delta(t+2m)<=4−c。

結論與檢核

取Psi=C(t+2m)、C>=a，攤銷成本<=ac+b+C(4−c)<=b+4C。同一C還須大到涵蓋extract-min的指標常數，固定一次即可；總帳靠非負Psi及初態0望遠鏡求和，不依賴c的機率分布。

19-E05 不能用二進位degree陣列界

推導

降鍵使id16被cut為根，id15原未標記，因失一child而標記，不再cut。extract移除id16後，原根1仍有degree4，總節點15，且只有一根，所以consolidate不會再改變其degree。

結論與檢核

floor(log2 15)=3，陣列只含0..3卻要放degree4，會越界。這是由合法操作歷史產生的反例；Fibonacci下界只保證degree4的子樹至少F6=8，沒有保證至少2^4=16。可用floor(log_phi n)的保守界或動態擴充degree槽位。

19-E06 相同最小鍵下的指定Handle刪除

推導

若另一節點y原已為INT_MIN，x降到同值不一定更新min到x；extract-min可能回傳y，錯刪handle。抽象−∞須嚴格小於所有普通鍵，不能直接拿型別邊界冒充。

結論與檢核

若x非根，cut x到root list並對舊父cascading-cut。接著移除這個指定根x，將其children提升為根、清mark，consolidate並重算min。只使x的handle失效，其他相同key節點identity保留。前段cut攤銷O(1)，指定根移除與extract-min同樣依degree/根數勢能得O(log(n+1))。

19 費氏堆積 解答 (續)

原創驗算程式附錄: cascade_cut_state.py

```
"""Evaluate both cascade-cut branches without merging their conclusions."""

from dataclasses import dataclass


@dataclass
class Node:
    name: str
    parent: object = None
    marked: bool = False


def cascade(root_is_g):
    root_list = []
    x, p, g = Node("x"), Node("p", marked=True), Node("g")
    root_list += [x, p]

    for node in (x, p):
        node.parent = None
        node.marked = False

    if not root_is_g:
        g.marked = True
    return root_list, {node.name: node.marked for node in (x, p, g)}


def run():
    nonroot_list, nonroot_marks = cascade(False)
    root_list, root_marks = cascade(True)
    assert [node.name for node in nonroot_list] == ["x", "p"]
    assert nonroot_marks == {"x": False, "p": False, "g": True}
    assert [node.name for node in root_list] == ["x", "p"]
    assert root_marks == {"x": False, "p": False, "g": False}
    print("PASS cascade nonroot_g=True root_g=False")
    return nonroot_marks, root_marks


if __name__ == "__main__":
    run()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

19 費氏堆積 解答 （續）

原創驗算程式附錄: fibonacci_checks.py

```
"""Logical Fibonacci-heap state model; Python lists do not model pointer costs."""
from dataclasses import dataclass, field
from random import Random


@dataclass(eq=False)
class Node:
    identity:int
    key:int
    parent:object=None
    children:list=field(default_factory=list)
    mark:bool=False


class Heap:
    def __init__(self):
        self.roots=[]
        self.nodes={}
        self.minimum=None
        self.next_id=0
        self.events=set()

    def insert(self,key):
        x=Node(self.next_id,key)
        self.next_id+=1
        self.nodes[x.identity]=x
        self.roots.append(x)
        if self.minimum is None or x.key<self.minimum.key:
            self.minimum=x
        return x

    def consolidate(self):
        degrees={}
        for x in list(self.roots):
            d=len(x.children)
            while d in degrees:
                y=degrees.pop(d)
                if y.key<x.key:
                    x,y=y,x
                y.parent=x
                y.mark=False
                x.children.append(y)
                d+=1
            degrees[d]=x
        self.roots=list(degrees.values())
        self.minimum=min(self.roots,key=lambda x:x.key,default=None)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

19 費氏堆積 解答 (續)

```
    def cut(self,x,parent):
        parent.children.remove(x)
        x.parent=None
        x.mark=False
        self.roots.append(x)
        self.events.add("cut")

    def cascading_cut(self,y):
        parent=y.parent
        if parent is not None:
            if not y.mark:
                y.mark=True
                self.events.add("mark-nonroot")
            else:
                self.cut(y,parent)
                self.events.add("cascade")
                self.cascading_cut(parent)
        else:
            self.events.add("stop-at-root")

    def decrease(self,x,new_key):
        assert new_key<=x.key and self.nodes[x.identity] is x
        x.key=new_key
        parent=x.parent
        if parent is not None and x.key<parent.key:
            self.cut(x,parent)
            self.cascading_cut(parent)
        if self.minimum is None or x.key<self.minimum.key:
            self.minimum=x

    def extract_root(self,x):
        assert x.parent is None
        self.roots.remove(x)
        for child in x.children:
            child.parent=None
            child.mark=False
            self.roots.append(child)
        del self.nodes[x.identity]
        self.consolidate()
        return x.key

    def pop_min(self):
        if self.minimum is None:
            return None
        return self.extract_root(self.minimum)

    def erase(self,x):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

19 費氏堆積 解答 (續)

```
        parent=x.parent
        if parent is not None:
            self.cut(x,parent)
            self.cascading_cut(parent)
        return self.extract_root(x)

    def potential(self):
        return len(self.roots)+2*sum(x.mark for x in self.nodes.values())

    def validate(self):
        seen=set()
        def visit(x,parent):
            assert x.identity not in seen
            seen.add(x.identity)
            assert self.nodes[x.identity] is x and x.parent is parent
            assert parent is None or parent.key<=x.key
            assert parent is not None or not x.mark
            size=1+sum(visit(child,x) for child in x.children)
            f0,f1=0,1
            for _ in range(len(x.children)+2):
                f0,f1=f1,f0+f1
            assert size>=f0
            return size
        for root in self.roots:
            visit(root,None)
        assert seen==set(self.nodes)
        assert self.minimum is None if not self.nodes else self.minimum.key==min(x.key for x
in self.nodes.values())
        return sorted(x.key for x in self.nodes.values())


def cascade_example(nonroot_g):
    heap=Heap()
    for key in range(65):
        heap.insert(key)
    assert heap.pop_min()==0
    target=next(x for x in heap.nodes.values()
                if len(x.children)>=2 and x.parent is not None
                and (x.parent.parent is not None)==nonroot_g)
    a,b=target.children[:2]
    g=target.parent
    ids=(a.identity,b.identity,target.identity,g.identity)
    heap.decrease(a,-10)
    assert target.mark
    before=heap.potential()
    heap.decrease(b,-11)
    assert target.parent is None and not target.mark
    assert g.mark==nonroot_g
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

19 費氏堆積 解答 （續）

```
    heap.validate()
    return ids,before,heap.potential()


def main():
    heap=Heap()
    for i in range(9):
        heap.insert(i)
    heap.pop_min()
    assert len(heap.roots)==1
    assert heap.roots[0].key==1 and len(heap.roots[0].children)==3
    assert heap.validate()==list(range(1,9))
    degree_counterexample=Heap()
    handles=[degree_counterexample.insert(i) for i in range(17)]
    assert degree_counterexample.pop_min()==0
    degree_counterexample.decrease(handles[16],-1)
    assert degree_counterexample.pop_min()==-1
    assert len(degree_counterexample.nodes)==15
    assert len(degree_counterexample.roots[0].children)==4
    degree_counterexample.validate()
    examples=[cascade_example(False),cascade_example(True)]
    rng=Random(1920)
    heap=Heap()
    operations=0
    for _ in range(2400):
        action=rng.randrange(4)
        if action==0 or not heap.nodes:
            heap.insert(rng.randrange(-200,201))
        elif action==1:
            x=rng.choice(list(heap.nodes.values()))
            heap.decrease(x,x.key-rng.randrange(50))
        elif action==2:
            expected=min(x.key for x in heap.nodes.values())
            assert heap.pop_min()==expected
        else:
            x=rng.choice(list(heap.nodes.values()))
            old=x.key
            assert heap.erase(x)==old and x.identity not in heap.nodes
        heap.validate()
        operations+=1
    print(f"PASS Fibonacci_logical_operations={operations} consolidation_root1_degree3
n15_degree4_counterexample=True cascade_examples={examples} events={sorted(heap.events)}")


if __name__=="__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

19 費氏堆積 解答 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study